



California State Polytechnic University Pomona
DEPARTMENT OF ELECTRONIC SYSTEMS ENGINEERING TECHNOLOGY
ETE3501: FEEDBACK AND CONTROL SYSTEMS

HALF TILT: TILTING MECHANISM WITH 1 AXIS, BALANCING A
MARBLE

Presented by:

Valeria Berrospid
Andy Guerrero
Samuel Arreaza
Christian Trujillo
Joanna Escobar
Philip Sorila

TABLE OF CONTENTS:

INTRODUCTION (Everyone)	3
BACKGROUND/DISCUSSION (Valeria)	3
THEORY (Philip & Sam)	5
DESIGN AND BUILD (Philip)	5
Testing Area (Joanna).....	6
Lever Assembly (Joanna).....	6
Axle and Motor (Joanna).....	8
Stands (Joanna and Andy).....	10
TESTING (Everyone)	12
Ultrasonic Sensor Testing (Valeria).....	13
PID Controller (Andy & Christian).....	14
Issues Resolved While Testing (Joanna).....	17
RESULTS (Valeria)	18
Transfer Function of the System (Andy).....	18
BILL OF MATERIALS (Joanna)	22
CONCLUSION (Everyone)	23
REFERENCES	24



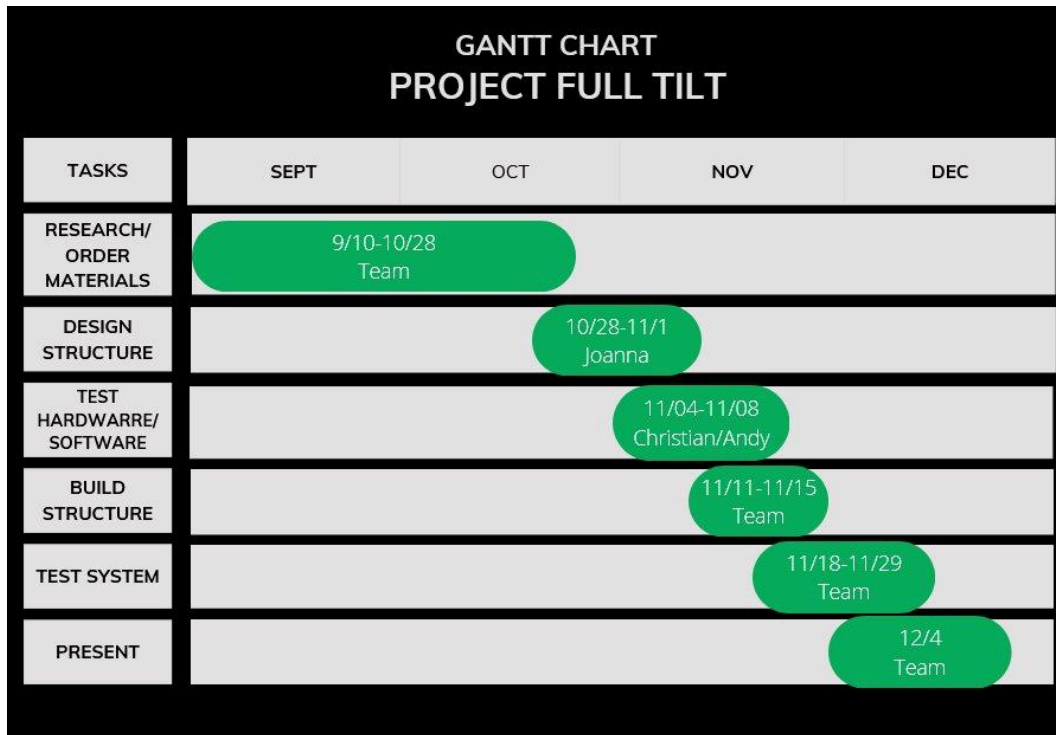
Left to Right: Valeria, Andy, Samuel, Christian, Joanna, Philip (middle)

INTRODUCTION (Everyone)

Our objective for this project is to effectively implement a control system with feedback to a lever arm in order to stabilize a moving spherical object and bring it to equilibrium. We chose to take on this objective because it will help us to better understand how hardware and software interact with each other to create a physical control system.

BACKGROUND/DISCUSSION (Valeria)

A balancing-ball system is a common, yet complex system that illustrates how a practical control system can work. We set up the following timeline to ensure we were progressing in a timely manner:



After receiving feedback from our Quad Chart presentation, we each took up the following roles: Joanna was in charge of research and 3D modeling, Valeria researched the data sheets for the chosen parts and organized the team meetings, Sam focused on the design of our balancing-ball control system, Andy and Christian were in charge of coding, and Philip focused on the physical construction. During testing, the whole team was present to help out with ideas to fine tune the control system as well as helping each person with their roles.

First, we researched the system to see what resources on the subject we could find. We especially focused on what sort of sensor would be the most suited for the system. Included in this research were strain gauges, accelerometers, and servo motor encoders. After feedback from Dr. Boskovich, we decided to go with: an HC-SR04 Ultrasonic Sensor, an Inland Blue 9G Servo Motor, and an Arduino UNO R3.

We wanted a motor that had enough torque strength to rotate the lever arm but still be low enough to reduce the effects of oscillation on the system. That's where the 9G micro servo motor became a part of interest to us for having a torque of 2.5kg/cm with a maximum angle rotation of 180 degrees. Its small size was also of interest because the extra weight added to the system would have smaller effects than if we chose a bigger motor.

To measure distance, we initially thought about a LiDAR sensor since some group members were already familiar with the sensor. Eventually, it was revealed that since our lever arm was to be 30 cm in length, a LiDAR sensor would not benefit the project because the minimum distance is measured in meters. In search for a smaller range, the HC-SR04 Ultrasonic Sensor was found to have a range of between 2 cm to 450 cm, enough to cover the length of our lever arm in small enough unit measurements. The way this sensor works is one lens transmits a signal that bounces off an object and returns to the receiver lens. The time it takes to receive the

transmission is then used to measure the distance. For this project, the sensor will detect the glass marble's distance from the center of the lever.

With the information collected from the sensor, an Arduino UNO R3 was selected to be used as our microprocessor to program the tilting feature using the distance recorded from the ultrasonic sensor. Using an Arduino helped limit our coding time, allowing for there to be more focus on testing values for the angle rotations and Kp, Ki, and Kd values of the PID controller that would ensure a steady-state system.

THEORY (Philip & Sam)

The balancing ball system utilizes a coded PID in order to achieve equilibrium for the test object. According to the description on Digikey's YouTube video: "What is a PID Controller | Digikey", a PID allows for a system to compensate for its own output when error is present. We knew that we would need a closed-loop system for this project, as by its very nature, the system's next output depends on the past position of the test object, in this case a glass marble. Also, in herding the marble towards equilibrium, we would need to make sure that the system didn't go overboard and destabilize the test object again by accident. As also described by Digikey's video description, the Integration and Derivative components of a PID controller work to keep the output in check, and make sure that the system is always heading towards steady-state; in our case, equilibrium of the test object.

We also decided on giving the apparatus a similar appearance to that of a teeter-totter. This was in order to simplify the design of the machine, and provide a more direct and linear approach to motivating the system. Most of the designs we saw—including the design made by Electronoob in his YouTube video—featured a motorized gear and arm, attaching to the servo on one side and a bottom edge of the wooden test area on the other. We decided against this actuation system in order to reduce complexity and potential for error in the design of the arms or gear. Instead, we decided to mount the motor directly on the systems' pivot point, the axle. That way, whenever the motor is actuated, its entire output will be sent to the lever, and we would not have to compensate for losses in torque or linear movement.

DESIGN AND BUILD (Philip)

The physical build of the project took place over several days. The build is made up of a few sections: the testing area, the lever assembly, the axle and motor, and the stands. The lever itself is made of a lightweight wooden block. The testing area consists of a channel for the test object to roll across before it reaches its steady state, as well as the microprocessor and sensors that will drive the behavior of the system. The axle and motor consist of a servo motor connected to a plastic dowel about $\frac{1}{2}$ an inch thick. This dowel is then attached to the bottom of the lever assembly in order for the motor to move the lever back and forth.

Finally, we have the stands. The stands serve to elevate the testing area to allow for tilting and movement. Some of the items we chose to use, such as the axle, were repurposed from other items. Everything else was 3D printed because it would be too expensive and time consuming to find the exact size we need and would be more practical to design the specific shape and size by hand. The testing area was also made by hand to allow us better control over the variables inherent in the project. The following subheadings go into more detail about how the different areas were constructed.

Testing Area (Joanna)

Our testing setup consists of an ultrasonic sensor, a microprocessor, and a glass marble as the testing object. Our ultrasonic sensor is placed at one end of the lever arm and connects to one of the analog inputs on the Arduino for the microprocessor code to use the distance being detected. From there, we were able to connect the micro servo motors to the Arduino. We chose to use a marble as our testing material because they are readily found and manufactured in various diameters and styles. The specific marble we chose was about an inch in diameter. We chose this specific marble because we felt that it best fit the scale, in both size and weight, of our testing area and apparatus.

Lever Assembly (Joanna)

The lever assembly is one section made of a variety of bonded parts. The most important testing area of the project is the lever itself. It was made of a wooden block because wood was readily available, light, and easy to work with. The testing area was made on one face of the block. This was done by sanding out a shallow channel in the center of the wooden block. The texture to the channel was beneficial for keeping the test object from moving left and right on the y-axis, as opposed to forwards and backward on the x-axis.

An I-shaped stabilizer for the bottom of the lever (Lever Plate) and a connector between the stabilizer and dowel (Lever Connector) were 3D printed out of PLA. The Lever Plate was screwed onto the bottom face of the wooden lever in order to ensure that the lever did not become disengaged from the axle at any point during the testing. There were two different iterations of the Lever Plate, one which had much more “empty space” cut out of the beam to keep the weight of the system down. However, we decided that the strength of the Plate and strength of the connection between the Plate and axle would take priority.

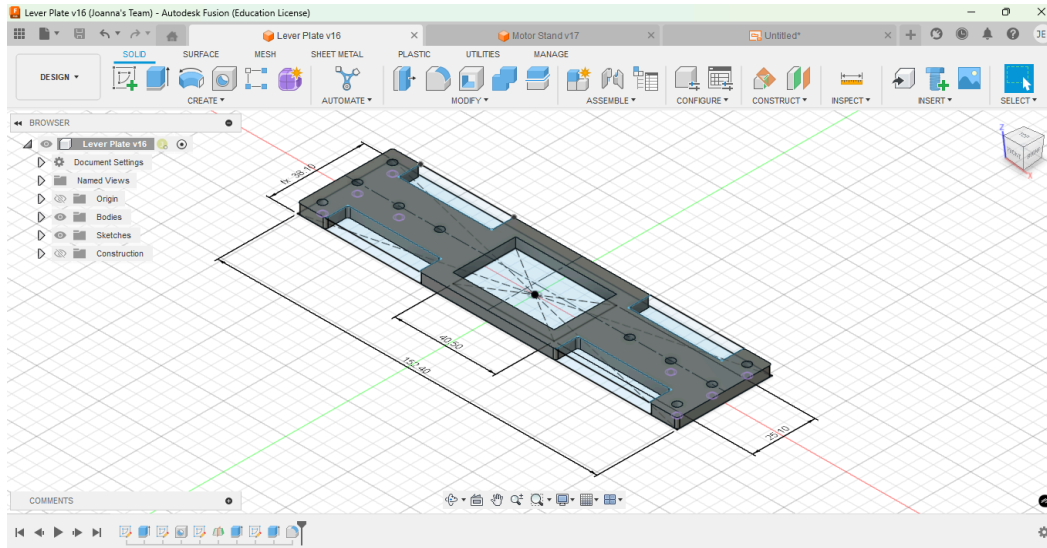


Image: 3D model of Lever Plate, as was designed using Fusion 360

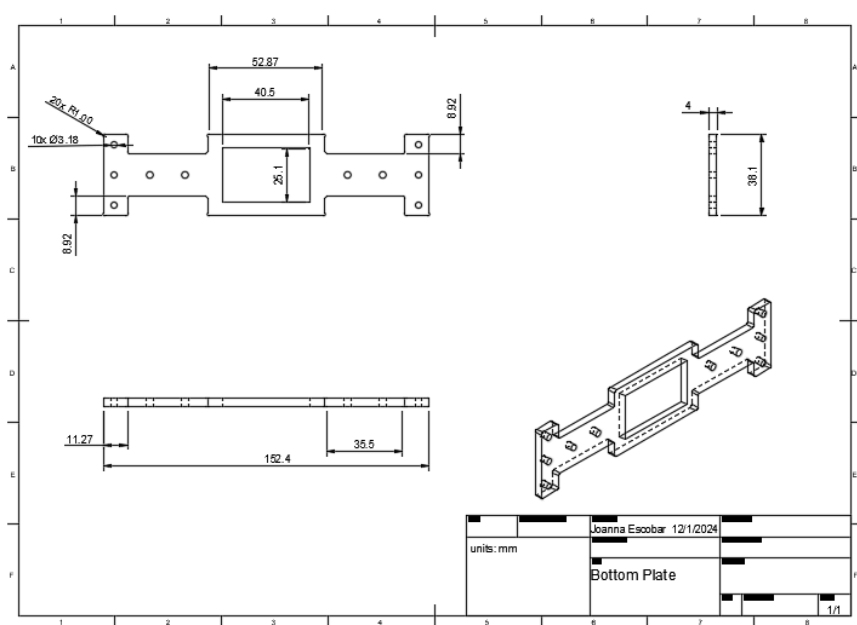


Image: Dimensional Sketch of the Lever Plate

The Lever Connector was designed to fit in the square opening in the middle of the plate. This was done partially to make the Connector and Plate easier to print, and partially to allow for experimentation with different Lever Plate designs before a final was selected. This part was designed to be placed directly on the axle, with minimal clearance between the two, and be held on with an adhesive. The adhesive that was used was the Gorilla brand Mounting Putty, in order to hold everything in place, while still allowing for some sort of flexibility in testing. The flat section of the Lever Connector was scored and superglued to the Lever Plate for a firm bond.

The superglue took a minimal time to dry and was very effective at bonding 3D parts, so we decided to use superglue for a majority of those joints.

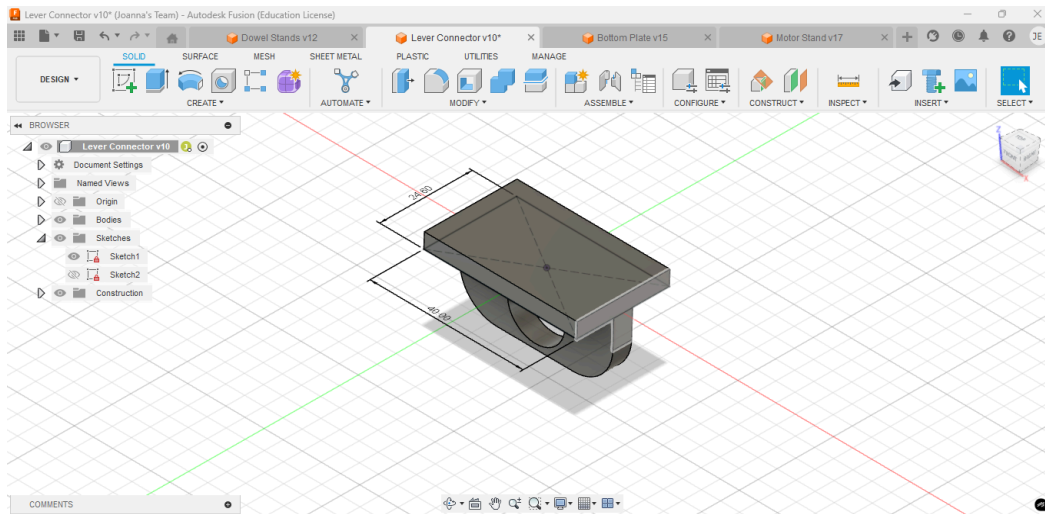


Image: 3D Model of Lever Connector, as designed in Fusion 360

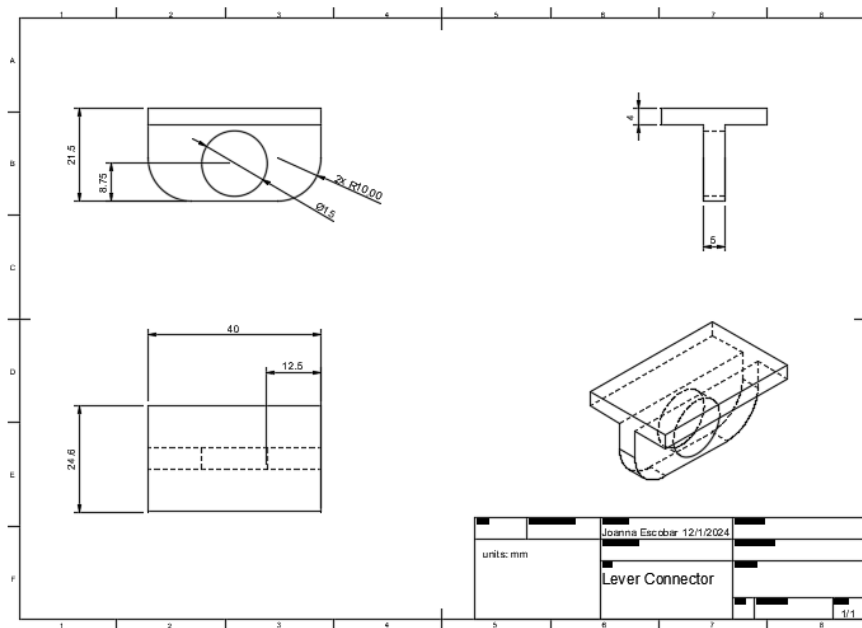


Image: Dimensional Sketch of Lever Connector

Once the Lever Plate, Lever Connector, and the wooden lever were all connected, they were affixed to the axle and Motor system.

Axle and Motor (Joanna)

As previously stated, the Inland Blue 9G Servo Motor was chosen for its small footprint and torque capability because we felt that the servo chosen would be more than enough to handle

the torque generated by our system. The axle itself was a hollowed plastic tube about 14.5mm in diameter and 45cm long. For efficiency's sake, we decided we used a premade plastic dowel, rather than 3D printing our own. We connected these two together by 3D printing a “washer” disk that would screw into one end of the dowel, as well as the motor's arms.

The washer we ended up using is pictured below. Multiple different versions of the disk were printed in order to give the team multiple choices to pick from. We decided on this version because it allowed for the most secure connection to the dowel. The screws chosen were wide enough to catch the plastic of both the disk and the object they were being screwed into, making the hold of the screw stronger. We screwed the washer into one end of the dowel, and into the servo motor's arms.

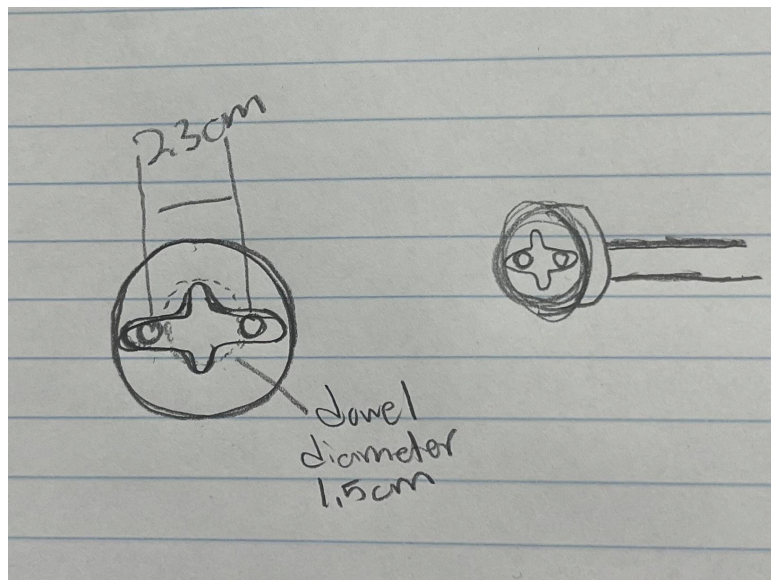


Image: Concept Drawing of Disk

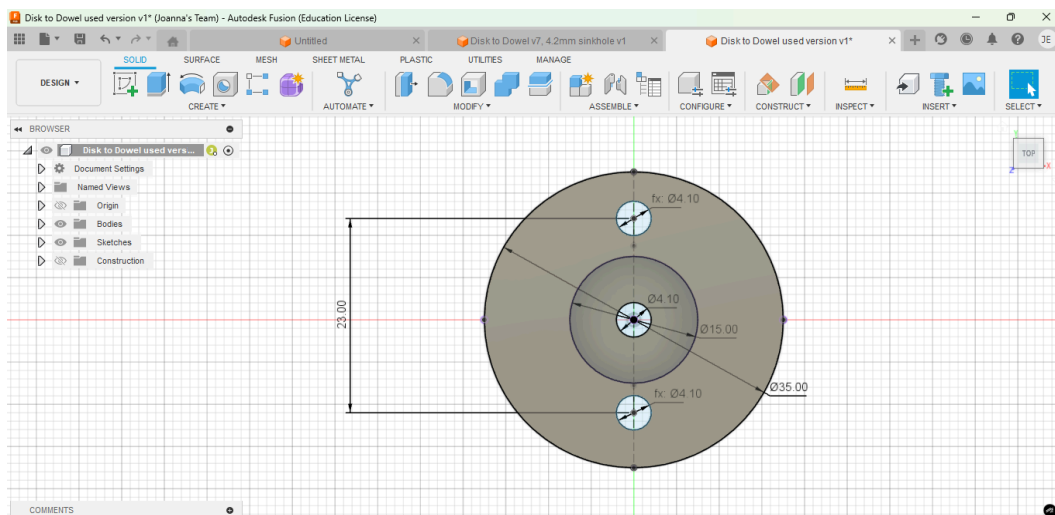


Image: 3D model of the washer connector disk

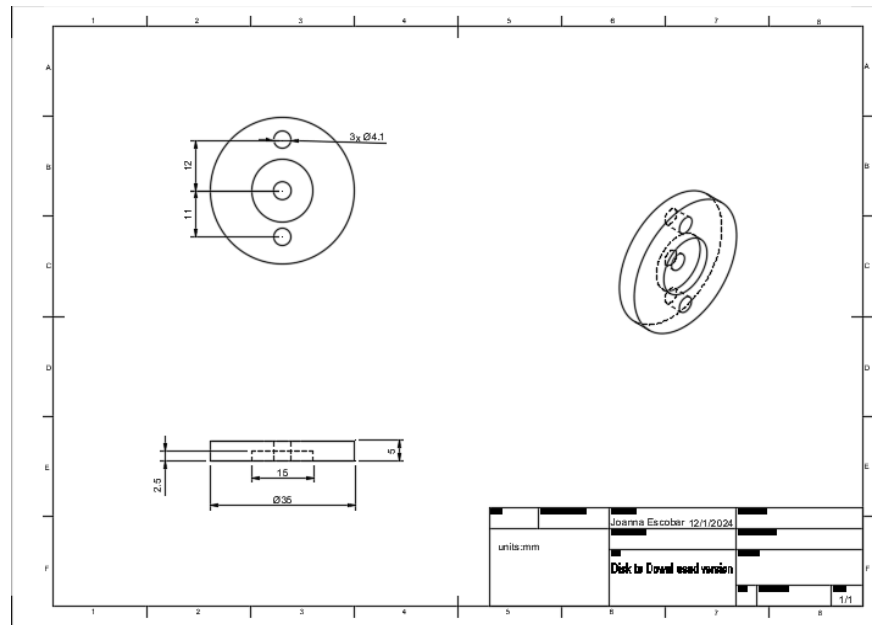


Image: Dimensional sketch of the disk

Stands (Joanna and Andy)

Assembling the lever and the stands to a platform was the next step. The dowel was then placed into the circular cutouts of the Dowel Stands.

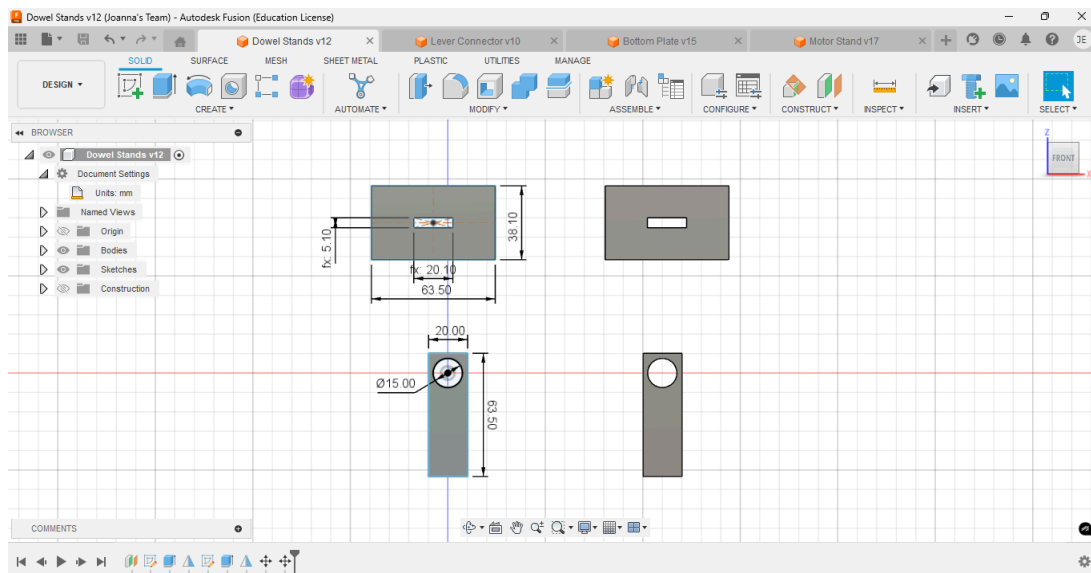


Image: 3D model of the Dowel Stand

The Dowel Stands had been designed with about 0.5mm clearance around the entire circumference of the dowel. This was in order to let the dowel turn freely, but to keep linear

movements to a minimum if there was any sudden movement in the servo motor's response. We did this in order to limit any additional values it might add to the transient response of the system. At the same time, we also wanted to keep the tolerances of the Dowel Stands large enough that the axle would be able to turn freely. We didn't want to add excessive friction into the system and possibly change the transient response of the system or have to compensate for excessive friction throughout our coding and calculations. Everything was glued onto a foam board with tacky glue to keep the spacing of the stands constant and to eliminate any accidents or accidental variables that might be introduced into the system.

A stand for the motor was also made at this time (called Motor Stand). Because the motor was attached directly to the dowel, it would need to be suspended off the ground and supported to keep gravity from overtaxing the motor or changing the torque it was able to exert on the system. That meant that we would need some sort of apparatus to hold the motor up to the right height and keep the body of the motor in one orientation as the arms rotate. The stand was made to hold the motor in one spot with little to no lateral movement. It also had a wide base, meant to keep the center of gravity of the system low and prevent the stand from getting picked up or tipped over from the force of the motor moving the rest of the system.

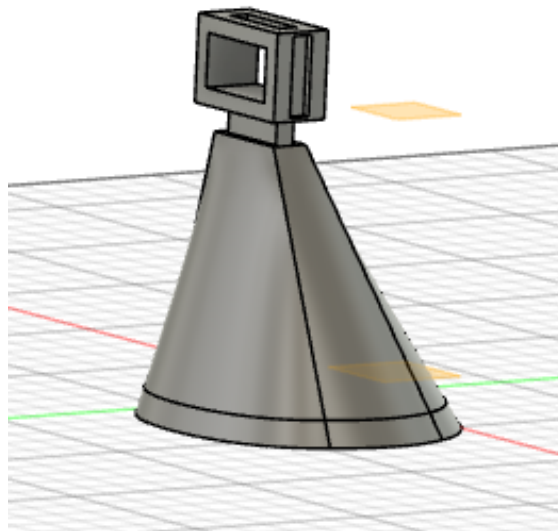


Image: 3D model of the Motor stand

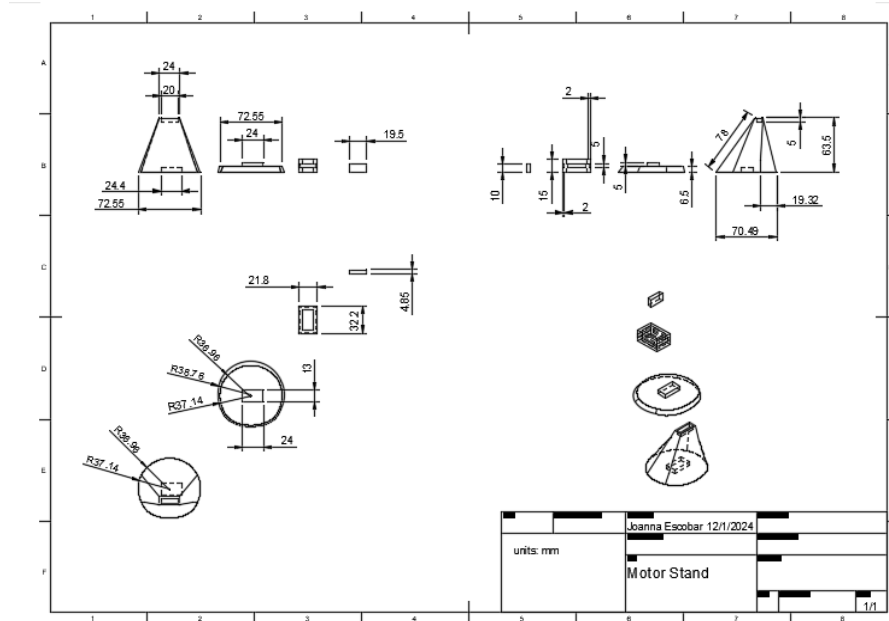


Image: Dimensional sketch of the motor stand

After completing the initial construction phase of the project, the next step involved programming the Arduino UNO R3 to control the servo motor with distance measurements from an ultrasonic sensor. The developed code used the sensor's functionality to dynamically adjust the servo motor's angle in response to real-time proximity data.

- First version of code was utilized to test ultrasonic sensor response
- Second version implemented PID controller
- Third version was the second version but tweaked
- The third version had an appropriate response from the system, which balanced the ball appropriately.

After having the code up and running, there were some final adjustments that needed to be made. The motor could not lift the wooden plank effectively due to the amount of mass on it. To fix this, more than half of the board was sawed off, making the board lighter and easier for the motor to control.

TESTING (Everyone)

In our first attempt to use our ESP32 WROOM 32 microcontroller for our project, we ran into issues with our COM ports. Ultimately, we switched to a microprocessor, the Arduino UNO R3. Using the Arduino IDE, the code developed had elements based off of a YouTube creator named Electronoob, whose concept of a ball balance was similar to our project but used a different construction and a different feedback device. The code had to first test out the functionality of the ultrasonic sensor.

Ultrasonic Sensor Testing (Valeria)

Here is the available code that tests the ultrasonic sensor and its ability to sense the marble and objects in general to move the servo motor:

```
int inputPin=4; // define ultrasonic signal receiver pin ECHO to D4
int outputPin=5; // define ultrasonic signal transmitter pin TRIG to D5
void setup()

{
  Serial.begin(9600);
  pinMode(inputPin, INPUT);
  pinMode(outputPin, OUTPUT);
}

void loop()
{
  digitalWrite(outputPin, LOW); delayMicroseconds(2);
  digitalWrite(outputPin, HIGH); // Pulse for 10µ s to trigger ultrasonic detection
  delayMicroseconds(10);
  digitalWrite(outputPin, LOW);
  int distance = pulseIn(inputPin, HIGH); // Read receiver pulse time
  distance= distance/58; // Transform pulse time to distance
  Serial.println(distance); //Output distance
  delay(50);
}
```

The ultrasonic sensor operates with two key pins:

- **TRIG Pin:** sends ultrasonic sound waves
- **ECHO Pin:** Detects the returning signal after the waves bounce off

The Arduino measures the time difference between the sending of the pulse and receiving the echo to calculate the distance to the object, in this case, the marble. This is done using the speed of sound formula:

$$Distance (cm) = \frac{Pulse Duration (\mu s) * 0.034}{2}$$

This formula and the sensor's continuous distance measurements enabled the system to gather real-time proximity data.

Based on the distance measured:

- **Close Proximity:** When an object is close, the servo motor positions itself at an angle that allows the ball to roll away from the sensor.
- **Farther Objects:** As the object moves farther away, the servo motor adjusts its angle proportionally so the ball doesn't fall off the balance..

- **No Object Detected:** The servo motor remains stationary or resets to a default position if there isn't an object detected.

This behavior creates a dynamic system where the servo motor's position correlates to the distance of the detected object. However, this code had the motor move at slow speeds, and that would not be viable for our project, as the motor needs to move a wooden board at decent enough speeds to control a marble. This prototype code was to see if the sensor could adjust the motor with the marble shifting its distance.

PID Controller (Andy & Christian)

After testing the sensor, the new version of the code implements a PID controller that adjusts the servo motor's angle based on the sensor's distance. This version of the code takes inspiration from the code provided by a video found on YouTube made by Electronoob. His project concept is similar to ours, with the difference of using a different build and feedback approach.

In the excerpt below, the pins are initialized for the sensor, as well as the object used to control the servo motor in the code. Other values, such as the distance, distance error, and PID gain values, are present.

```
#include <Servo.h>

//////////////////// Inputs/outputs //////////////////////
const int trigPin = 9; // Ultrasonic sensor TRIG pin
const int echoPin = 10; // Ultrasonic sensor ECHO pin
Servo myservo;          // Create servo object to control servo

//////////////////// Variables //////////////////////
float distance = 0.0;
float elapsedTime, time, timePrev; // Variables for time control
float distance_previous_error, distance_error;
int period = 50; // Refresh rate period of the loop is 50ms

|

//////////////////// PID Constants //////////////////////
float kp = 8; // Proportional gain
float ki = 0.2; // Integral gain
float kd = 3100; // Derivative gain
float distance_setpoint = 21; // Desired distance in cm
float PID_p, PID_i, PID_d, PID_total;
```

The values used for the PID gains are for testing purposes and will be adjusted later, along with the code. The `distance_setpoint` variable in the code is the distance from the sensor to the middle of the balance in centimeters. This might have to be adjusted in the future as well.

The K_p value in our code represents the proportional response of the servo motor to the marble's position. It adjusts the motor angle in response to the distance error, which is the difference between the marble's current position and the desired position. The K_i value represents the cumulative error over time. This helps to correct minor persistent errors by the sum of errors in each loop, gradually shifting the marble toward the desired position. The K_d value is the derivative response, which reacts to the rate of change of the marble's position. It adjusts the motor based on the marble's speed, which helps dampen any overshoot and stabilize the system.

```
void setup()
{
  Serial.begin(9600);
  myservo.attach(3);      // Attach the servo to pin 3
  myservo.write(125);     // Set initial position
  pinMode(trigPin, OUTPUT); // Set TRIG pin as output
  pinMode(echoPin, INPUT);  // Set ECHO pin as input
  time = millis();
}
```

This code sets up the servo motor to be attached to pin 3 on the Arduino UNO and the TRIG pin as an output pin, and the ECHO pin as an input. The `myservo.write(125);` line adjusts the servo motor to an initial position of 125°. This will be adjusted later to our system specifications.

The code below makes up the majority of the implementation of the PID controller. It adjusts the servo motor's position based on real-time measurements of the distance of the marble from the ultrasonic sensor. This would ensure steady execution by using a timing function maintaining proper measurement intervals. The function calculates the distance error by comparing the measured distance to the set point where the marble is supposed to be. Then, it uses the K_p , K_i , and K_d values to constrain the marble to stay in the middle. The PID output is then mapped to the servo motor's range of motion and constrained to safe limits. The servo motor reacts dynamically, which tilts the rail to correct the marble's position and prevent it from rolling off. For debugging, the function is set up to output the marble's current distance from the sensor and the servo motor's angle on the Serial Monitor for the Arduino IDE. Ultimately, the goal is to maintain the marble in the center of the rail through continuous and accurate adjustments.

```
void loop()
{
    if (millis() > time + period)
    {
        time = millis();

        // Gets the distance from ultrasonic sensor
        distance = getDistance();

        // Calculates PID errors
        distance_error = distance_setpoint - distance;
        PID_p = kp * distance_error;

        PID_d = kd * ((distance_error - distance_previous_error) / period);

        // Integral term only applies for small errors
        if (-3 < distance_error && distance_error < 3)
        {
            PID_i += (ki * distance_error);
        } else
        {
            PID_i = 0;
        }

        // Total PID output
        PID_total = PID_p + PID_i + PID_d;

        // Map PID output to the servo angle range
        PID_total = map(PID_total, -150, 150, 0, 150);

        // Constrains servo angle to safe range
        if (PID_total < 20) PID_total = 20;
        if (PID_total > 160) PID_total = 160;

        myservo.write(PID_total + 30); // Adjusts the servo angle
        distance_previous_error = distance_error;

        // Debugging information
        Serial.print("Distance: ");
        Serial.print(distance);
        Serial.print(" cm, Servo Angle: ");
        Serial.println(PID_total + 30);
    }
}
```

This code is the function that gets the distance from the sensor:

```
// Function to get distance from ultrasonic sensor
float getDistance() {
    long duration;
    float distance_cm;

    // Send a 10us pulse to TRIG pin (adjust to specs)
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    // Reads the ECHO pin, and calculates distance (adjust to specs)
    duration = pulseIn(echoPin, HIGH);
    distance_cm = duration * 0.034 / 2; // Convert to cm

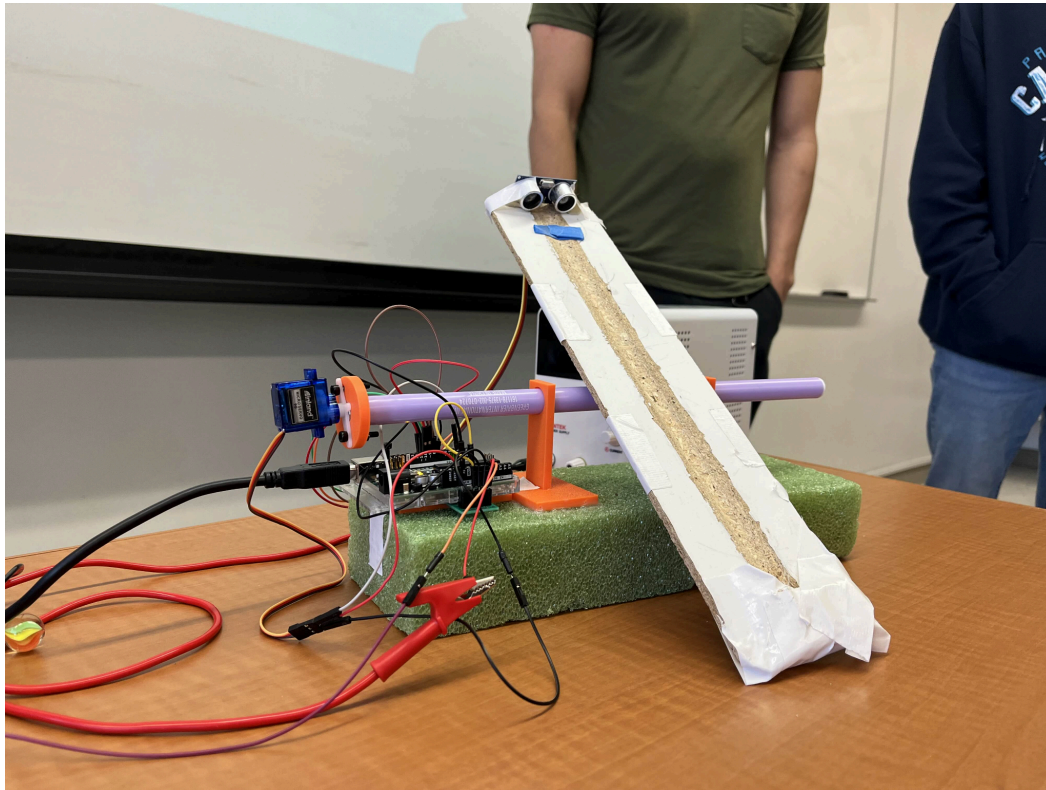
    return distance_cm;
}
```

Issues Resolved While Testing (Joanna)

When testing, we discovered that the wooden plank was slightly too heavy, causing the servo to stall. We first attempted to adjust the code to compensate for that, but eventually decided to remove volume from the lever instead. From there, we reattached the Lever Plate and Lever Connector, and readjusted the code to reflect the changed system.

In addition, the motor stand was not heavy enough to be able to keep the motor from moving. While this did not derail the project, it is something that would be addressed in further versions of this system, were we to continue improving on it.

RESULTS (Valeria)



In the end, our balance ball system was able to work, but has a few imperfections. For starters, while the marble is able to balance itself on the lever, it levels out on one side of the lever rather than the middle. We also noticed that there was little to no rotation on one side of the lever compared to the other side that had the attached sensor. After testing, we determined this was because of the motor's position not being perfectly level with the lever during the attachment process. The improper mounting of the motor was causing a disturbance to the servo motor that affected the torque strength on the lever.

Another issue that remained with the end product was how the system would stop responding when the marble was too close to the sensor. We resolved this issue by using some tape as a stopper, but the sensor would be picking up the location of the stopper rather than the marble. Even with these issues that remained at the end, our ball-balancing system was able to function as intended by at least leveling the ball on the lever.

Transfer Function of the System (Andy)

We managed to derive a MATLAB code that analyzes our control system. The servo motor can be viewed as a first-order system, and the seesaw as a second-order system with damping and natural frequency. The open-loop transfer function mixes the controller, motor, and seesaw dynamics, while the closed-loop transfer function has feedback which determines the system's overall behavior. The code calculates and displays both open-loop and closed-loop transfer functions, which simulates our step response to evaluate the system's performance. We

also have an s-plane plot to analyze our stability. This process shows how the PID controller interacts with the physical components to achieve as stable of a response as possible from our system.

```
% PID Controller
Kp = 8; % Proportional Gain
Ki = 0.2; % Integral Gain
Kd = 3100; % Derivative Gain

% servo time constant (estimated from specs)
tau_s = 0.05; % (time constant ≈ 0.05s)
G_servo = tf(1, [tau_s 1]);

% seesaw values (assumed)
k = 1; % Gain for the seesaw
zeta = 0.7; % Damping ratio
wn = 5; % Natural frequency (rad/s)

% T.F of seesaw
G_seesaw = tf(k, [1 2*zeta*wn wn^2]);

% T.F of PID controller
s = tf('s');
G_PID = Kp + Ki/s + Kd*s;

% open-loop
G_open = G_PID * G_servo * G_seesaw;

% closed-loop
G_closed = feedback(G_open, 1);
```

```

% Outputs transfer functions
disp('Servo Transfer Function (G_servo):');
[n_servo, d_servo] = tfdata(G_servo, 'v'); %
fprintf(' Numerator: %s\n', mat2str(n_servo));
fprintf(' Denominator: %s\n', mat2str(d_servo));

disp('Seesaw Transfer Function (G_seesaw):');
[n_seesaw, d_seesaw] = tfdata(G_seesaw, 'v');
fprintf(' Numerator: %s\n', mat2str(n_seesaw));
fprintf(' Denominator: %s\n', mat2str(d_seesaw));

disp('PID Controller Transfer Function (G_PID):');
[n_pid, d_pid] = tfdata(G_PID, 'v');
fprintf(' Numerator: %s\n', mat2str(n_pid));
fprintf(' Denominator: %s\n', mat2str(d_pid));

disp('Open-Loop Transfer Function (G_open):');
[n_open, d_open] = tfdata(G_open, 'v');
fprintf(' Numerator: %s\n', mat2str(n_open));
fprintf(' Denominator: %s\n', mat2str(d_open));

disp('Closed-Loop Transfer Function (G_closed):');
[n_closed, d_closed] = tfdata(G_closed, 'v');
fprintf(' Numerator: %s\n', mat2str(n_closed));
fprintf(' Denominator: %s\n', mat2str(d_closed));

% step response
figure;
step(G_closed);

```

```

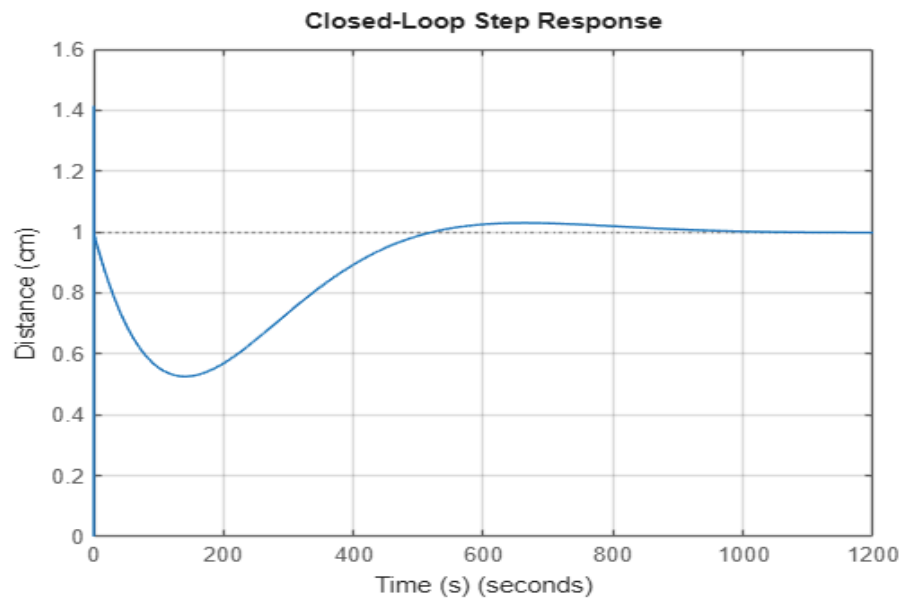
grid on;
title('Closed-Loop Step Response');
xlabel('Time (s)');
ylabel('Distance (cm)');

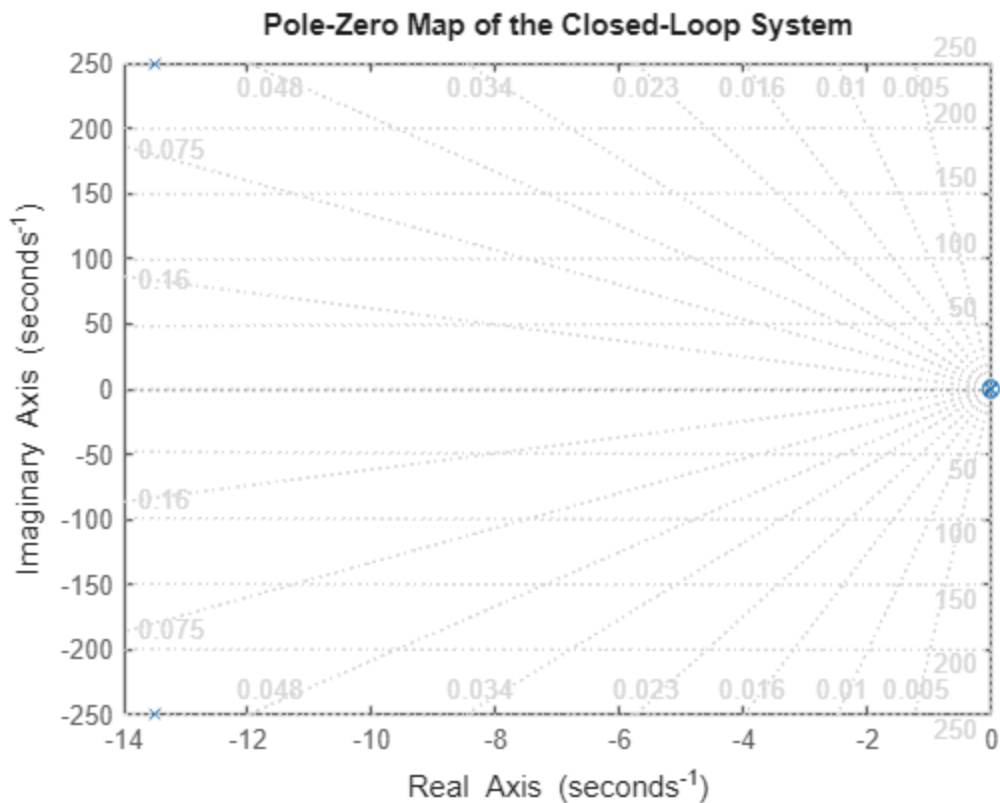
% pole-zero map
figure;
pzmap(G_closed);
grid on;
title('Pole-Zero Map of the Closed-Loop System');

```

Here are our outputs:

```
Servo Transfer Function (G_servo):  
  Numerator: [0 1]  
  Denominator: [0.05 1]  
Seesaw Transfer Function (G_seesaw):  
  Numerator: [0 0 1]  
  Denominator: [1 7 25]  
PID Controller Transfer Function (G_PID):  
  Numerator: [3100 8 0.2]  
  Denominator: [0 1 0]  
Open-Loop Transfer Function (G_open):  
  Numerator: [0 0 3100 8 0.2]  
  Denominator: [0.05 1.35 8.25 25 0]  
Closed-Loop Transfer Function (G_closed):  
  Numerator: [0 0 3100 8 0.2]  
  Denominator: [0.05 1.35 3108.25 33 0.2]
```





Our closed-loop transfer function ends up being:

$$G(s) = \frac{3100s^2 + 8s + 0.2}{0.05s^4 + 1.35s^3 + 8.25s^2 + 25s + 0.2}$$

This was derived by the use of our PID gain values, the specifications of the servo motor, and the observation of the behavior of our seesaw mechanism. Our closed-loop transfer function combines all of these functions into one.

BILL OF MATERIALS (Joanna)

- Arduino Uno R3 - \$26.30 (pre-owned by Joanna)
- Wooden Lever (raw materials) - \$0.00
- MC-SR01 - \$4.50
- 3D Printed
 - Lever Plate - \$0.26
 - Lever Connector - \$0.21
 - Dowel Stands - \$0.54
 - Motor Stand - \$1.38

- Disks - \$0.72
- Dowels - \$1.25
- Additional Wooden Kits (raw materials) - \$2.50
- Sanding Materials - \$1.25
- Inland Blue Micro Servo Motor (3 pack) - \$10.99
- Floral Foam - \$4.99
- Adhesives
 - Tacky Glue - \$2.99
 - Gorilla Glue Brand Wall Mounting Putty - \$5.49
 - Superglue - \$1.25
 - Duct Tape - \$5.99

CONCLUSION (Everyone)

Using spare wooden planks, we carved a path, or railing, for our marble. It was necessary to cut off a significant portion of its initial height because the servo motor was unable to turn its mass. Mounting putty was used to attach a 3D-printed axle to the wooden railing. The axle and two other 3D-printed stands have holes to secure a dowel, where more mounting putty was used to connect to the axle with the wooden railing. A 3D-printed washer connects the end of a dowel to the hands of our servo motor. The center screw hole that attaches to the dowel was altered to match the bevel of our screws. The holes on the servo motor hands were not big enough so we drilled our own holes. A 3D printed stand secures the position of the motor so that it shall not move while turning the dowel, and a block of floral foam secures the 3D-printed stands. Our initial microcontroller was switched out for an Arduino UNO R3 because of compatibility issues with our laptop's drivers.

The materials and parts used are all either lightweight, inexpensive, or readily available. Our team members already had the majority of the parts, including the wood, a 3D printer, and multiple sensors to choose from. The only exceptions were the marbles, dowel, and adhesives.

With a functional and compatible microcontroller, we are able to receive distance measurements of the moving marble and calculate its speed, change of speed, and distance away from the center of the wooden railing. Based on the distance away from the center of the wooden railing, the servo motor will adjust its rotation and positioning so that the marble will remain at the center, or at least oscillate around it. Right now, the servo motor moves appropriately but the behavior is jittery and not smooth. Perhaps we can change the Kd value on the PID to smoothen the sudden change of rotation or continue to decrease the mass of the railing to decrease load and quickness of the rotation.

REFERENCES

- Digikey. (2023, Aug 21). *What is a PID controller? | Digikey* [Video]. YouTube.
<https://www.youtube.com/watch?v=tFVAaUcOm4I>
- Electrical4U. (2024, Apr 17). *Servo Motor: Definition, Working Principle, and Applications*
<https://www.electrical4u.com/what-is-servo-motor/>
- Electronoobs. (2019, July 14). *PID Balance+Ball | full explanation & tuning* [Video]. YouTube.
<https://www.youtube.com/watch?v=JFTJ2SS4xyA>
- GV Stories. (2023, May 22). *Self-Balancing Ball Control System Project | PID Control* [Video].
 YouTube. <https://www.youtube.com/watch?v=IOOin9FKW4c>
- Keyestudio Wiki. (2021, Apr 27). *KS038 Keyestudio HR-SR04 Blue Ultrasonic Module*
https://wiki.keyestudio.com/KS0328_Keyestudio_HR-SR04_Blue_Ultrasonic_Module
- Micro Center: Computers & Electronics. (2022, Mar 16). *Inland Blue 9G Servo*
<https://community.microcenter.com/kb/articles/659-inland-blue-9g-servo>
- Random Nerd Tutorials. (2024, June 11). *ESP32 with DC Motor and L298N Motor Driver – Control Speed and Direction*
[https://randomnerdtutorials.com/esp32-dc-motor-l298n-motor-driver-control-speed-direction/#:~:text=one%20DC%20motor-,Wiring%20a%20DC%20Motor%20to%20the%20ESP32%20\(L298N%20Motor%20Driver,motor%20driver%20to%20the%20ESP32](https://randomnerdtutorials.com/esp32-dc-motor-l298n-motor-driver-control-speed-direction/#:~:text=one%20DC%20motor-,Wiring%20a%20DC%20Motor%20to%20the%20ESP32%20(L298N%20Motor%20Driver,motor%20driver%20to%20the%20ESP32)